

UE24CS341A : Software Engineering

Lab 3: Component Modelling & Architectural Pattern Selection

Topic: Peer Skill Management & Mentorship Network

Pranav Somashekar

PES2UG24CS364

This submission evaluates candidate architectural styles for a Peer Skill Management and Mentorship Network, selects the most appropriate style, and presents a UML Component Diagram showing the resulting modules, interfaces, and dependencies.

Step 1 — Scenario Definition

System: Peer Skill Management & Mentorship Network — a platform where employees or students list the skills they have and the skills they want to learn, get automatically matched with suitable peer mentors, book and run mentorship sessions, and rate those sessions afterwards.

Core Functional Requirements

- Users can create and maintain a profile listing skills offered and skills sought.
- The system matches mentees to mentors based on skill overlap, availability, and past ratings.
- Users can book, reschedule, and complete mentorship sessions (in-person or virtual).
- The system sends notifications for match results, booking confirmations, and reminders.
- Mentees can rate and review mentors after each completed session.

Non-Functional Requirements & Key Challenges

- **Scalability:** matching and notification load grows independently of profile/session volume, especially during peak onboarding periods (e.g. start of a semester or quarter).
- **Availability:** booking and notifications must keep working even if the matching algorithm is being updated or is temporarily degraded.
- **Security:** profile data (contact details, skill history) and session records are personal data and need isolated access control per service.
- **Extensibility:** new matching strategies (e.g. AI-based recommendation) or notification channels (SMS, push) should be addable without redeploying the whole system.

Step 2 — Architectural Style Analysis

The three candidate architectural styles introduced in this lab are compared below against the scenario's functional and non-functional requirements.

Style	Structure	Pros (for this scenario)	Cons (for this scenario)
Layered	Horizontal layers: Presentation, Business, Data	Simple to understand and build quickly; clear separation of UI from logic from storage	Matching and notification load cannot scale independently; a single business layer becomes a bottleneck and a single point of failure for all features
Microservices	Small, independently deployable services, each owning its own data	Matching Service can scale separately from Notification or Profile services; a failure in one service (e.g. Notifications) does not block booking or profile updates; teams can evolve the matching algorithm independently	More operational complexity (multiple deployable units); inter-service calls add network latency; requires careful interface design for data consistency
Client-Server	Centralized server handling all requests from thin clients	Centralized control and simple data consistency	Single point of failure; the server becomes a scalability bottleneck under simultaneous matching, booking, and notification traffic; limited fault isolation between features

Step 3 — Architecture Selection & Component Identification

Selected Architecture: Microservices. The scenario's requirement for independently scaling features (matching load vs. booking vs. notifications) and its need for fault isolation (a notification outage should never block booking) make Microservices the strongest fit among the three candidates.

Identified Components

The lab brief supplies two anchor components for this system — the **Mentorship Session Service** and the **Notification Service** — reflecting the given Order Manager ↔ Payment Service pairing adapted to this domain (a session booking is this system's core transaction, and it must trigger a downstream notification). The remaining three components below were identified from the scenario's functional requirements to complete the required set of five.

#	Component	Responsibility	Source
1	Mentorship Session Service	Handles booking, rescheduling, and completion of mentorship sessions	Given
2	Notification Service	Sends match results, booking confirmations, and session reminders to users	Given

3	User Profile Service	Stores and manages user profiles: skills offered, skills sought, contact info	Identified
4	Skill Matching Service	Matches mentees to mentors based on skill overlap, availability, and rating history	Identified
5	Review & Rating Service	Collects post-session ratings and reviews, feeding scores back into matching	Identified

Step 4 — Component Diagram & Interfaces

The table below lists every interface shown in the component diagram, including the one given in the brief (Order Manager ↔ Payment Service, mapped here to Mentorship Session ↔ Notification/Review flows) plus the additional interfaces identified to fully connect all five components.

Provider → Consumer	Interface	Type	Technology
User Profile → Skill Matching	ProfileData	Assembly (ball/socket)	REST API
Skill Matching → Mentorship Session	MatchResult	Assembly (ball/socket)	REST API
Mentorship Session → User Profile	ValidateUser	«use» dependency	REST API
Mentorship Session → Notification	SessionEvents	Assembly (ball/socket)	Event Queue
Mentorship Session → Review & Rating	SessionRecord	Assembly (ball/socket)	REST API
Review & Rating → Skill Matching	RatingScore	Assembly (ball/socket)	Event Queue
Notification → User Profile	ContactInfo	Assembly (ball/socket)	REST API
External user → User Profile	ManageProfile	Provided (external)	REST / UI
External user → Skill Matching	FindMentor	Provided (external)	REST / UI
External user → Mentorship Session	BookSession	Provided (external)	REST / UI
External user Notification	SendAlert	Provided (external)	Email / SMS / Push

This satisfies the deliverable requirement of at least 5 components and at least 4 inter-component interfaces, using proper UML provided (ball) / required (socket) notation and «use» dependency notation where appropriate.

UML Component Diagram

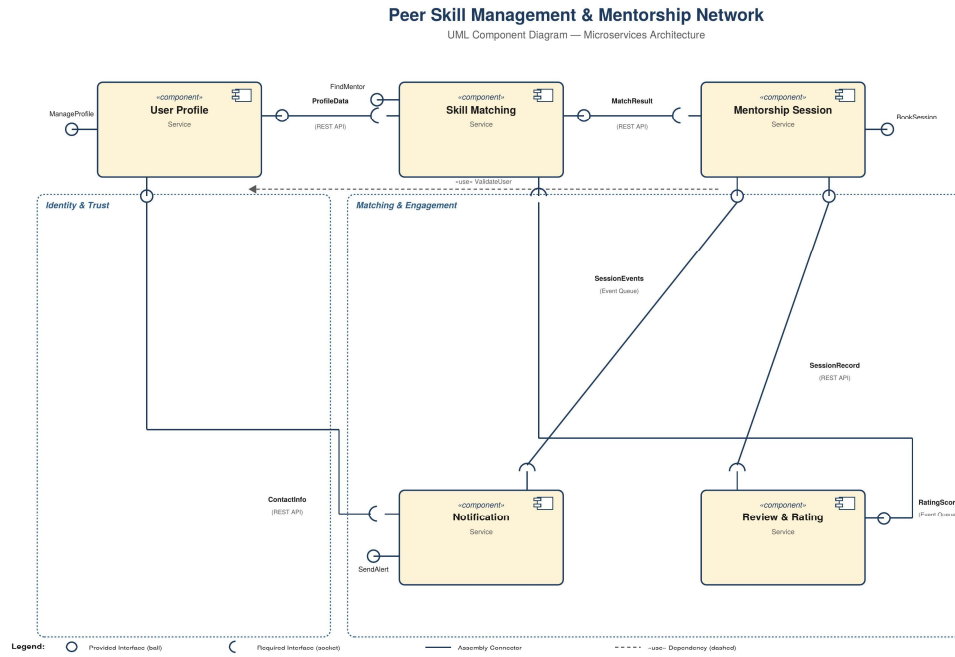


Figure 1. Component diagram for the Peer Skill Management & Mentorship Network, showing all five components, their provided/required interfaces, assembly connectors, and «use» dependencies, grouped into two logical subsystems (Identity & Trust; Matching & Engagement).

Written Justification

We chose **Microservices Architecture** for the Peer Skill Management & Mentorship Network System.

Architectural Choice

Microservices Architecture was selected over Layered and Client-Server architectures. Each core capability — user profiles, skill matching, session booking, notifications, and reviews — is implemented as an independently deployable service with its own data and a small, well-defined set of provided and required interfaces.

Reason 1: Independent Scaling Under Uneven Load

Skill matching is computationally heavier and spikes sharply during onboarding periods (e.g. the start of a semester), while profile updates and notifications have a steadier, lighter load. In a Layered or Client-Server design, all functionality shares one scaling unit, so the whole system must be over-provisioned to handle the matching spike. With Microservices, only the Skill Matching Service needs to scale out during peak matching periods, keeping infrastructure cost proportional to actual demand.

Reason 2: Fault Isolation Between Features

Mentorship booking is the platform's core value and must stay available even if a secondary feature fails. Because the Notification Service and Review & Rating Service are separate deployable units, an outage or slow dependency in notifications (e.g. an email provider being down) cannot block the Mentorship Session Service from completing a booking. In a Client-Server design, a single centralized server would make notification failures a potential single point of failure for the entire platform.

Security Advantage

Microservices allow each service to enforce access control scoped to only the data it owns. The User Profile Service is the sole owner of personal contact information and exposes it only through a narrow **ContactInfo** interface consumed by the Notification Service — the Skill Matching and Session services never receive raw contact details at all, reducing the blast radius if any single service is compromised. A monolithic Layered design, by contrast, typically gives the whole business layer direct database access to all user data.

Performance Benefit

Because the Review & Rating Service publishes **RatingScore** updates asynchronously through an event queue rather than a blocking synchronous call, the Skill Matching Service can consume updated ratings on its own schedule without ever waiting on the Review Service during a live match request. This asynchronous decoupling keeps the user-facing FindMentor and BookSession interfaces fast and responsive, even when the review pipeline is under heavy load.